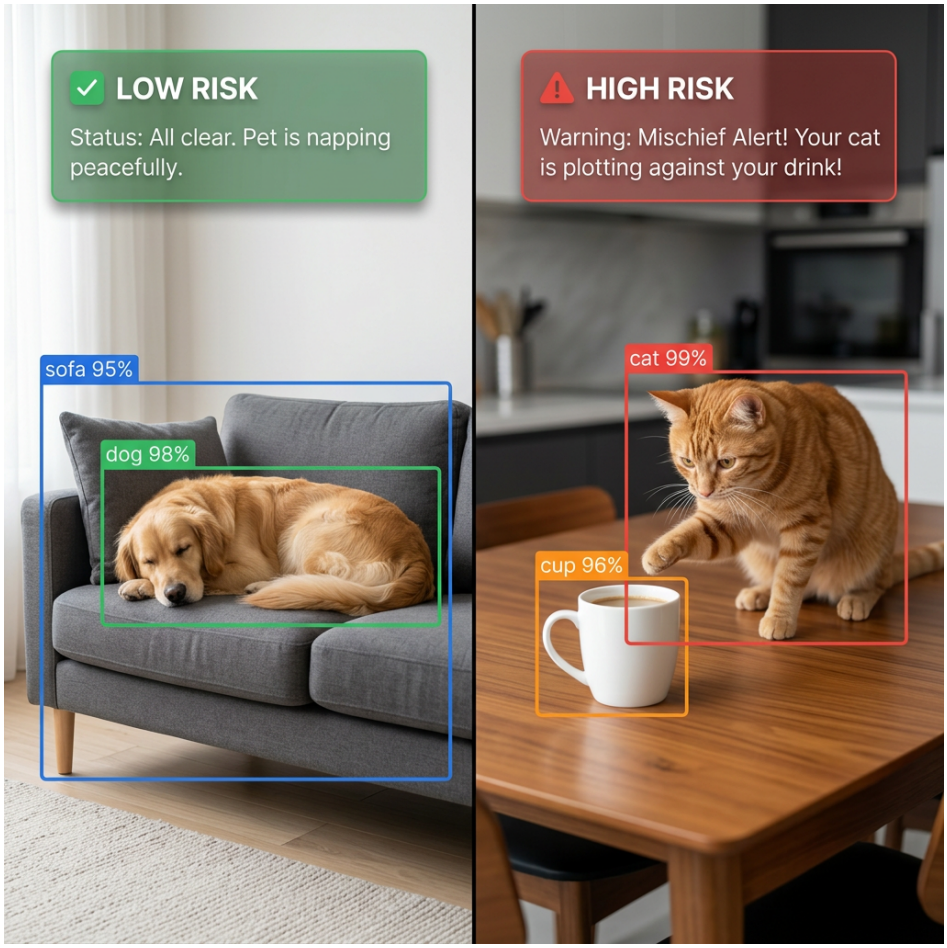
COMP4423 – Computer Vision

Project - The Pet Mischief Detector

[Deadline: 11:59 PM on Thursday, 30 April 2026]



1. Task Description

Is your cat about to knock over that vase? Is your dog chewing on your laptop cable? In this project, your group will build a system that can answer these questions. The core idea is to apply computer vision techniques to analyze a scene, understand the spatial relationships between pets and objects, and predict potentially mischievous scenarios.

Your mission is to build an end-to-end system that first analyzes an image to identify and locate pets and various household objects. It then uses this spatial information to calculate a "mischief score" or issue a context-aware warning. For example:

- **High Risk:** A cat is identified very close to a cup. Warning: "Mischief Alert! Your cat is plotting against your drink!"
- **Low Risk:** The area occupied by a dog is overlapping with a sofa. Status: "All clear. Pet is napping peacefully."

This is an open-ended problem. The specific logic for assessing risk, and the vision techniques you use to gather the necessary information, are up to you to design, implement, and justify. This project will challenge you to define a problem, curate a dataset, train a robust vision model, and apply your own creative logic to interpret the scene's context. Your solution should be based on the methods learned in this course or from your own research. You are encouraged to be creative. Your approach could be based on, but is not limited to:

- **Object Detection Models** (e.g., YOLO family, DETR, Faster R-CNN, SSD) to find the bounding boxes of entities.
- **Instance Segmentation Models** (e.g., Mask R-CNN, YOLACT) to get precise pixel-level masks, which may be better for judging contact or close proximity.
- **Combinations of Models** (e.g., using a foundation model like SAM for segmentation combined with a classification model).

2. Group Work Policy

This is a group project. Each group may have **up to 3 members**. Please indicate your group members' name and student IDs on the first page of the report. Group members will collaborate on the project implementation (code, model training, etc.).

However, the project report must be written and submitted by yourself, individually by each group member*. Despite the project being a group effort, your final grade may vary depending on the quality of your individual report. Each individual report must contain a section titled **"Statement of Contribution"** detailing your specific role and contributions to the project. Marks will be deducted if this section is missing.

**Despite the same project done by members in one group, the final grades may varies depending on the quality of the individual report.*

3. Tasks & Assessment

Task 1: Problem Definition & Data Curation (10 marks)

Your first task is to define the scope of your mischief detector and build a dataset for it.

- **Define Your Classes and Rules:**
  - Choose the classes you want to detect. You must include **at least one pet class** (e.g., cat, dog) and **at least five household object classes** that could be involved in mischief (e.g., cup, laptop, plant, vase, power_cable, etc).
  - Define a preliminary set of "mischief rules" based on the spatial relationships between these classes.
- **Curate Your Dataset:**
  - You are responsible for assembling a dataset for your chosen classes. We recommend filtering large, existing datasets like [COCO] or [Open Images] to extract images and annotations for your classes.
  - Alternatively, you may collect and annotate your own images, but be aware this is very time-consuming.
  - Your final curated dataset should be well-organized. Document its final size, class distribution, and any pre-processing steps you performed.

Task 2: Dataset Splitting & Formatting (10 marks)

Properly splitting your data is crucial for unbiased model evaluation.

- **Split the Data:** Programmatically split your curated dataset into training, validation, and test sets (e.g., 80%/10%/10%). Ensure this split is reproducible (e.g., by using a fixed random seed).
- **Format for Training:** Convert your dataset into the format required by your chosen detection framework (e.g., YOLO TXT, COCO JSON). Document this process and provide any conversion scripts.

*Task 3: Training/Fine-tuning a Vision Model (15 marks)**

You will fine-tune a model of your choice on the training set you created.

- **Select and Train a Model:** Choose a vision model and train it on your custom dataset.
- **Justify Training Choices:** In your report, clearly explain and justify your key training decisions (e.g., choice of model, pretrained weights, input resolution, data augmentation, optimizer, learning rate, etc).
- **Provide Training Evidence:** Submit training logs and/or plots of key metrics (e.g., loss, mAP) to show the model's learning progress. Your training process must be reproducible.

*Task 4: Mischief System Implementation & Evaluation (15 marks)**

This task has two parts: evaluating the model and demonstrating your complete mischief system.

- **Part A: Quantitative Detector Evaluation:** Evaluate your core vision model on your test set. You must report standard metrics appropriate for your chosen method (e.g., for object detection, report mAP@0.5 and mAP@[0.5:0.95]; for segmentation, you might report mask mAP or mIoU). Justify your choice of metrics.
- **Part B: Qualitative System Evaluation:**
    - Implement the "mischief logic" that takes the model's output (e.g., class labels, bounding boxes, segmentation masks) and generates a risk score AND a warning message.
    - In your report, provide visualizations from your test set that showcase your full system in action. Include examples of correctly identified high-risk scenarios, low-risk scenarios, and any failure cases. Justify why your logic is effective.

Task 5: Individual Report (50 marks)

Submit a comprehensive individual report detailing your entire process. A template will be provided, and all questions must be answered thoroughly. **This report must be your own work.** It must also include a **"Statement of Contribution"** section, clearly outlining the tasks you were responsible for within your group (e.g., "I was responsible for writing the data-splitting script and setting up the initial YOLOv8 training pipeline," "I focused on hyperparameter tuning and writing the evaluation code.").

Bonus (up to 10 marks):
Submissions demonstrating exceptional code quality, insightful analysis, high model accuracy, and a creative and effective mischief logic are eligible for a bonus. An excellent way to earn this bonus is by testing your model's real-world generalization:

- **Generalization Test:**
  1. Individually or as a group, collect **5-10 photos of your own home environments** (with or without pets).
  2. Run your complete "Pet Mischief Detector" system on these new images.
  3. In your report, analyze the results. How well does your system generalize to completely new images? What are its limitations? This analysis is crucial.
- The final grade for this assignment will be: `min(100, normal_grade + bonus)`.

Collaborate with Generative AI for problem-solving:** You are encouraged to use Generative AI to assist with implementation and debugging (e.g., dataset conversion, training scripts, evaluation or visualization utilities), but you must understand and verify the generated code and clearly describe in your report what AI helped with, what limitations or mistakes you observed, and how you corrected or improved the pipeline based on your own reasoning and testing. However, it's important to recognize that ***AI can make mistakes, so you should ***identify its limitations and areas for improvement*** in your report. An example prompt is provided at the end of this file for your reference.

4. Submission

Each group member must submit the following files individually:

1. Name your individual report as `Project_Report_<your_ID>_<your_name>.pdf`.
   e.g., `Project_Report_12345678d_CHAN_Dawen.pdf`
2. Compress your group's complete implementation files (code, scripts, etc.) into a `.zip` file and rename it using your own ID and name.
   e.g., `Project_Code_12345678d_CHAN_Dawen.zip`
3. Upload the above two files to the Blackboard system.

Warning:
If you are unable to complete the whole program, try to accomplish part of the tasks and make sure it can run successfully.

Any wrong file naming and submission will be given a ZERO mark in this assignment.

- The deadline for this assignment is **11:59 PM on Thursday, 30 April 2026**.
- Late submission penalty
- 10% is deducted for each day that the work is late. The penalty will be applied up to a maximum number of three days after and including the submission deadline day. After three days the work will be marked at zero.

This assignment is individual work. All work must be done on your own. Plagiarism is a serious offence. Copying code from web resources is prohibited as well. Any plagiarism case (for both the copier and the copiee) will be given a ZERO mark in this assignment.

5. Example Prompts

https://genai.polyu.edu.hk/ (GPT-5.1)

Task 3:

```
# Task description
I am doing the COMP4423 Pet Mischief Detector project. I have already
curated and split my dataset into training/validation sets, and I
have a `dataset.yaml` file ready for YOLOv8. Now I need to fine-tune
a model.

# What you should do
Please write a Python training script for me that:
(1) Uses the Ultralytics YOLOv8 library.
(2) Loads a pretrained model (e.g., yolov8n.pt).
(3) Starts training on my custom dataset defined in the `dataset.yaml`
file.
(4) Allows me to easily configure key hyperparameters like image
size, batch size, and number of epochs.
(5) Saves the training logs and the best model weights to a `runs/`
directory.

Please include clear comments where I might need to change file paths
or model names.
```

Task 4:

```
# Task description
I am doing the COMP4423 Pet Mischief Detector project. I have
already fine-tuned a YOLOv8 model (`best.pt`) on my custom dataset
of pets and household objects. Now I need to build the complete
mischief detection system.

# What you should do
Please write a Python script that:
(1) Loads my custom-trained YOLOv8 model (`best.pt`).
(2) Runs inference on an input folder of images.
(3) For each image, applies a custom "mischief logic" to the
detected bounding boxes. Please provide a placeholder function
`calculate_mischief(detections)` where I can implement my own rules
(e.g., checking proximity between 'cat' and 'cup').
(4) Saves the output images with both the bounding boxes and a "LOW
RISK" or "HIGH RISK" text label drawn on them based on the logic's
output.
```